Модуль сквозного шифрования трафика предназначен для защиты данных, передаваемых между терминалами сбора данных и сервером логистической компании по беспроводным каналам связи. Модуль выполняет генерацию мастер-ключа сервера, формирование производных ключей для каждого устройства через алгоритм HKDF-SHA256, шифрование команд алгоритмом AES-256-GCM с последующим расшифрованием и замером времени операций, имитацию атаки злоумышленника с попыткой расшифрования перехваченных данных чужим ключом, а также вывод результатов в виде таблиц в консоли, сохранение CSV-логов и построение графиков.

1. Объекты тестирования контейнера и подходы к их тестированию

Для программного средства «Модуль сквозного шифрования трафика» были выделены следующие объекты тестирования.

Алгоритм генерации ключей. Данный объект представляет собой программную функцию key_for_device, реализующую диверсификацию ключей устройств (процесс создания разных ключей для разных устройств из одного общего мастер-ключа) через алгоритм HKDF-SHA256 на основе мастер-ключа и идентификатора устройства, а также инициализацию мастер-ключа через secrets.token_bytes(32). Выбор данного объекта для тестирования обусловлен тем, что диверсификация ключей является критически важной для безопасности всей системы, так как компрометация ключа одного терминала сбора данных не должна позволить расшифровать трафик других устройств. Ошибка в работе HKDF приведёт к тому, что разные устройства получат одинаковые производные ключи, и защита потеряет смысл.

Подходы к тестированию: структурное тестирование для проверки внутренней логики функции — одинаковые идентификаторы устройств должны давать одинаковые производные ключи, а разные идентификаторы — разные ключи; модульное тестирование для изолированной проверки длины выходного ключа, которая должна составлять ровно 32 байта; тестирование на

основе спецификации для проверки соответствия функции требованию диверсификации ключей, заявленному в техническом задании.

Алгоритм шифрования и защиты от атаки. Объект объединяет две взаимосвязанные функции. Первая encrypt_and_decrypt, выполняющая шифрование данных, передаваемых между терминалом сбора данных и сервером, алгоритмом AES-256-GCM с генерацией случайного nonce, последующее расшифрование и замер времени обеих операций. Вторая attacker_decrypt, имитирующая попытку злоумышленника расшифровать перехваченные данные с использованием неверного ключа. Данный объект реализует ключевую защитную функцию контейнера: ошибка в шифровании приводит к нарушению конфиденциальности передаваемых данных, а ошибка в имитации атаки приводит к тому, что реальная угроза перехвата не будет обнаружена.

Подходы к тестированию: тестирование на основе спецификации для проверки корректности шифрования — расшифрованные данные должны полностью совпадать с исходными, время выполнения операций не должно превышать 50 миллисекунд; тестирование на основе состояний для проверки двух возможных исходов в функции имитации атаки — успешное расшифрование с правильным ключом и отклонение попытки с чужим ключом через исключение InvalidTag; нагрузочное тестирование для оценки поведения алгоритма при обработке пакетов разной длины и подтверждения стабильности временных характеристик; негативное тестирование для проверки реакции системы на некорректные входные данные, включая пустой список пакетов и повреждённый шифротекст.

Табличное представление результатов. Данный объект представляет собой модуль, отвечающий за формирование трёх таблиц в консоли с использованием библиотеки Pandas и сохранение результатов легитимной передачи и попыток атак в CSV-файлы encryption_log.csv и attacker_log.csv. Выбор данного объекта для тестирования связан с необходимостью обеспечения корректного представления результатов работы контейнера:

ошибка в структуре таблиц или кодировке файлов приводит к неверной интерпретации данных администратором системы.

Подходы к тестированию: ручное тестирование для визуальной проверки корректности форматирования таблиц в консоли, выравнивания колонок и читаемости данных; автоматизированное тестирование для сравнения структуры DataFrame с ожидаемой — количество строк должно соответствовать количеству тестовых пакетов, названия колонок должны совпадать с заданными; интеграционное тестирование для проверки взаимодействия модуля обработки данных с файловой системой — CSV-файлы должны создаваться после каждого запуска программы и содержать актуальные результаты.

Визуализация результатов. Данный объект включает функции построения и сохранения двух графиков с использованием библиотеки Matplotlib: график времени шифрования и расшифрования chart_time.png и график размера пакетов до и после шифрования chart_size.png, на которые выводятся подписи значений. Выбор данного объекта для тестирования связан с необходимостью обеспечения корректного представления результатов работы контейнера конечным пользователям: ошибка визуализации способна привести к неправильной интерпретации данных.

Подходы к тестированию: ручное тестирование для визуальной проверки корректности построения графиков — легенда должна соответствовать отображаемым данным, цвета столбцов и линий должны различаться, подписи значений должны быть читаемы и расположены без наложений; регрессионное тестирование для подтверждения неизменности формата выходных графиков после внесения модификаций в код функций шифрования или обработки данных.

2. Типы ошибок в контейнере

Опираясь на классификацию аномалий по IEEE 1044-2009 «Standard Classification for Software Anomalies», а также на показатели качества программного обеспечения, определённые в ГОСТ Р 56920-2016, для

разработанного модуля сквозного шифрования трафика были выделены следующие типы ошибок, представленные в таблице 1.

Таблица 1 — Классификация типов ошибок в контейнере

Категория ошибки	Описание потенциального дефекта в контексте кода	Критичность
Ошибка в данных	Переменная PACKETS инициализирована пустым списком. Цикл for p in PACKETS не выполняет ни одной итерации, список encrypted_pairs остаётся пустым, последующий цикл for p, (nonce, ct) in zip(PACKETS, encrypted_pairs) также не выполняется. Программа завершается без вывода таблиц, без сохранения CSV-файлов и без построения графиков. Сообщения об ошибке при этом не предусмотрено.	Незначительная
Ошибка в данных	Поле data команды ТСД содержит данные в формате, отличном от ожидаемой строки UTF-8, например бинарную последовательность. Вызов text.encode("utf-8") либо выбрасывает исключение UnicodeDecodeError, либо преобразует данные с искажениями. В первом случае программа завершается аварийно, во втором — шифруется некорректная строка, и после расшифрования результат не совпадает с исходной командой склада.	Серьёзная
Ошибка в логике	В функции attacker_decrypt нарушен порядок возвращаемых значений в блоках try и except InvalidTag. При возникновении исключения InvalidTag, означающего несовпадение тега аутентификации, функция должна возвращать значение False, однако в блоке except ошибочно возвращается True. В результате система сообщает об успешном чтении данных злоумышленником, тогда как в действительности данные защищены. Ошибка приводит к ложной интерпретации состояния безопасности модуля.	Критическая

Ошибка в стандарте	Повторное использование одного и того же значения nonce с одним ключом шифрования. Стандарт NIST SP 800-38D для режима AES-GCM требует уникальности nonce при каждом шифровании. Нарушение данного требования приводит к тому, что две одинаковые команды порождают идентичные шифротексты, что позволяет злоумышленнику обнаружить повторяющиеся команды и анализировать паттерны работы склада. Кроме того, повторное использование nonce в режиме GCM создаёт предпосылки для криптоанализа и компрометации ключа.	Критическая
Ошибка в интерфейсе	Вызов df.to_csv() или plt.savefig() завершается исключением PermissionError при отсутствии у процесса прав на запись в целевую директорию. Криптографические операции при этом выполняются успешно, таблицы и графики формируются в памяти, однако сохранить результаты на диск не удаётся. Работа модуля остаётся незафиксированной.	Серьёзная
Ошибка в интерфейсе	При сохранении CSV-файлов не указан параметр encoding='utf-8-sig'. В результате при открытии файлов encryption_log.csv и attacker_log.csv в Microsoft Excel русскоязычные названия колонок отображаются нечитаемыми символами.	Незначительная
Ошибка в интерфейсе	Отсутствие в окружении необходимых библиотек cryptography, matplotlib, pandas и numpy вызывает исключение ImportError на этапе импорта. Модуль завершается аварийно до выполнения какой-либо содержательной операции. Данный дефект классифицируется как ошибка интерфейса, поскольку затрагивает взаимодействие модуля со средой выполнения.	Серьёзная

3. Типы тестов для проверки контейнера

Типы тестов для проверки модуля сквозного шифрования трафика выбраны исходя из специфики его работы: генерация криптографических ключей, шифрование данных алгоритмом AES-256-GCM и имитация атаки перехвата. При выборе учитывались как функциональные требования технического задания, так и лекционный материал, описывающий классификацию видов тестирования.

Модульное тестирование. Данный тип тестирования применяется для изолированной проверки отдельных функций модуля. Для функции key_for_device выполняются следующие проверки: при передаче одного и того же идентификатора устройства функция должна возвращать одинаковый ключ, при передаче разных идентификаторов ключи должны различаться, длина возвращаемого ключа должна составлять ровно 32 байта. Для функции encrypt_and_decrypt проверяется, что после шифрования и последующего расшифрования данные восстанавливаются без искажений, а время выполнения обеих операций не превышает 50 миллисекунд. Для функции attacker_decrypt: возврат значения False при попытке расшифрования с чужим ключом, возникновение исключения InvalidTag при несовпадении тега аутентификации. Изоляция функций при тестировании позволяет точно установить, в каком именно компоненте находится дефект и избежать ситуации, когда ошибка в одной функции маскируется корректной работой другой.

Интеграционное тестирование. Интеграционное тестирование направлено на проверку корректности взаимодействия между компонентами модуля при их последовательном вызове. Контролируется, что ключ, сгенерированный функцией key_for_device, без ошибок принимается объектом AESGCM и используется при шифровании в encrypt_and_decrypt. Проверяется, что результаты каждого прохода цикла по PACKETS правильно накапливаются в списках results и encrypted_pairs, а во втором цикле элементы этих списков сопоставляются попарно без смещений и потери данных. Отдельно проверяется запись итоговых данных: после завершения программы

в рабочей директории должны появиться файлы encryption_log.csv и attacker_log.csv, количество строк в которых совпадает с числом тестовых пакетов, а названия колонок соответствуют заданным. Дополнительно верифицируется согласованность данных между таблицами: количество записей со статусом OK в таблице легитимной передачи должно равняться общему количеству пакетов, количество записей «ДАННЫЕ ЗАЩИЩЕНЫ» в таблице попыток перехвата также должно равняться общему количеству пакетов.

Функциональное тестирование. Функциональное тестирование проверяет соответствие модуля требованиям технического задания. На тестовом наборе команд ТСД запускается полный цикл работы программы, и её выходные данные сравниваются с ожидаемыми. Контролируется выполнение следующих требований: диверсификация ключей согласно ТЗ — ключи разных устройств не должны совпадать; защищённость трафика — злоумышленник с чужим мастер-ключом не должен иметь возможность прочитать перехваченные данные; производительность — время шифрования и расшифрования не должно превышать 50 миллисекунд; логирование — результаты работы должны сохраняться в структурированные CSV-файлы. Если хотя бы одно из перечисленных требований не выполнено, модуль не может считаться готовым к эксплуатации.

Нагрузочное тестирование. Нагрузочное тестирование оценивает устойчивость модуля при изменении объёма обрабатываемых данных. Выполняется шифрование команд ТСД разной длины: от минимальной (7 байт), команда OP:PICK, до максимальной (34 байта) полная команда с артикулом, количеством и локацией, а также искусственно увеличенных строк размером 1 КБ и 10 КБ. Для каждого размера фиксируется время шифрования и расшифрования. Модуль считается прошедшим нагрузочное тестирование, если при любом размере пакета время операций остаётся стабильным и не превышает 50 миллисекунд. Дополнительно выполняется многократный

прогон одного и того же набора пакетов для выявления возможной деградации времени при длительной работе.

Негативное тестирование. Негативное тестирование проверяет поведение модуля в ситуациях, выходящих за рамки нормальной работы. Подаётся пустой список команд, и программа должна завершиться без аварийной остановки, желательно с выводом предупреждения об отсутствии данных. В функцию расшифрования передаётся намеренно испорченный шифротекст, ожидается выброс исключения InvalidTag и возврат значения False. Используется nonce длиной не 12 байт, а 8 или 16, тем самым проверяется, что модуль либо корректно обрабатывает нестандартный nonce, либо информирует об ошибке. Имитируется отсутствие прав на запись в текущую директорию, программа не должна завершаться аварийно, а должна вывести в консоль причину, по которой не удалось сохранить файлы. Проверяется запуск модуля в окружении без библиотек cryptography, matplotlib, pandas и numpy — ожидается сообщение о необходимости установки зависимостей.

Регрессионное тестирование**.** Регрессионное тестирование выполняется после каждого внесения изменений в исходный код модуля, чтобы убедиться, что после доработок существующие функции продолжают работать без сбоев. Оно включает повторный запуск модульных и интеграционных тестов после любого изменения в функциях шифрования, генерации ключей или логирования. Совпадение новых результатов с эталонными значениями, полученными до внесения изменений, подтверждает, что код можно безопасно интегрировать в основную ветку проекта. Для автоматизации регрессионного тестирования используется фреймворк pytest, что позволяет выполнять проверку при каждом обновлении кода без ручного запуска.

4. Задачи группы тестировщиков контейнера

Анализ требований и верификация логики шифрования. Изучается техническое задание на модуль сквозного шифрования трафика, выделяются ключевые требования, подлежащие проверке. Особое внимание уделяется

функции диверсификации ключей: необходимо убедиться, что алгоритм HKDF-SHA256 корректно формирует уникальные ключи для каждого устройства. Проверяется функция шифрования и расшифрования: данные после полного цикла преобразования должны совпадать с исходными, а время операций не должно превышать 50 миллисекунд. Отдельно анализируется функция имитации атаки: при неверном ключе должно выбрасываться исключение InvalidTag и возвращаться значение False.

Проверка корректности накопления и сохранения результатов. После обработки каждого тестового пакета результаты должны корректно заноситься в список results, а пары nonce и шифротекст — сохраняться в encrypted_pairs. Контролируется, что во втором цикле элементы списков сопоставляются без смещений. Кроме этого, осуществляется проверка сохранения CSV-файлов: encryption_log.csv должен содержать все записи легитимной передачи с корректными статусами, attacker_log.csv — все записи попыток перехвата с результатом «ДАННЫЕ ЗАЩИЩЕНЫ».

Тестирование визуализации и консольного вывода. Три таблицы в консоли должны формироваться без смещения колонок и потери данных, все числовые значения обязаны соответствовать фактическим результатам измерений. Проводится проверка построения графиков: график времени шифрования и расшифрования должен содержать две линии с подписями значений, график размера пакетов до и после шифрования включает в себя парные столбцы с читаемыми метками. Легенда и заголовки обязаны присутствовать на обоих графиках.

Негативное тестирование. Проверяется поведение модуля при подаче некорректных входных данных: пустой список команд PACKETS, повреждённый шифротекст, nonce неверной длины. Модуль должен сохранять работоспособность и не завершаться аварийно. Отдельно контролируется реакция на отсутствие прав записи в директорию и на запуск в окружении без необходимых библиотек.

Разработка автоматизированных тестов. Реализуется набор тестов на базе фреймворка pytest, покрывающий ключевые функции модуля: key_for_device, encrypt_and_decrypt, attacker_decrypt. Обеспечивается возможность автоматического запуска тестов при внесении изменений в код. Автоматизируется регрессионное тестирование для проверки стабильности работы модуля после доработок.

Документирование дефектов и формирование отчётности. Все обнаруженные расхождения с ожидаемым поведением фиксируются в баг-репортах с указанием входных данных, ожидаемого результата, фактического результата и критичности. По завершении тестирования готовится итоговый отчёт с метриками качества и заключением о готовности модуля к эксплуатации.

5. Организация работ для проведения тестирования контейнера

Организация работ по тестированию модуля сквозного шифрования трафика выполнена в соответствии с процессами тестирования, описанными в ГОСТ Р 56920-2016 и лекционном материале. Этапы работ, их содержание и используемые инструменты представлены в таблице 2.

Таблица 2 — Организация работ по тестированию

Этап	Деятельность	Инструменты / Ресурсы
Планирование	Определение объектов тестирования, выбор типов тестов, распределение задач между участниками группы. Формирование перечня проверяемых требований ТЗ.	Стандарты: ГОСТ Р 56920-2016, IEEE 1044-2009. Документ: План тестирования.
Проектирование	Разработка тестовых сценариев для функций key_for_device, encrypt_and_decrypt, attacker_decrypt. Подготовка спецификаций тестов и процедур.	Среда: Replit. Артефакты: тест-кейсы для pytest.

	Создание набора тестовых команд ТСД.	
Подготовка среды	Настройка секретных ключей в Replit. Установка библиотек cryptography, matplotlib, pandas, numpy. Генерация тестовых значений мастер-ключей.	Платформа: SaaS Replit. ОС: Linux (Shell).
Выполнение	Прогон модульных тестов через pytest. Ручное выполнение интеграционных, нагрузочных и негативных тестов. Контроль вывода таблиц, проверка сохранения CSV и графиков.	Фреймворк: pytest. Shell: команды python.
Оценка результатов	Сравнение результатов с ожидаемыми. Проверка времени шифрования (≤ 50 мс). Анализ CSV-файлов и графиков. Классификация дефектов по IEEE 1044-2009.	Метрики: процент пройденных тестов, количество дефектов, среднее время шифрования.
Повторное и регрессионное тестирование	Повторная проверка исправленных дефектов. Прогон регрессионных тестов для подтверждения отсутствия побочных эффектов.	Фреймворк: pytest. VCS: Git.
Завершение	Составление итогового отчёта с метриками качества и заключением о готовности модуля. Фиксация версии кода в Git.	VCS: Git (GitHub).

6. План тестирования

6.1 Идентификатор документа

Параметр	Значение
Идентификатор	TSP-Encryption-Container-v1.0
Название	План тестирования модуля сквозного шифрования трафика
Дата выпуска	24.04.2026
Автор	Рощина А.В.
Статус	Финальный

6.2 Введение

Настоящий документ содержит план тестирования модуля сквозного шифрования трафика, разработанного для защиты данных, передаваемых между терминалами сбора данных и сервером логистической компании по беспроводным каналам связи. Модуль реализует генерацию криптографических ключей, шифрование команд алгоритмом AES-256-GCM, имитацию атаки злоумышленника с чужим ключом, сохранение результатов в CSV-файлы и построение графиков. Тестирование проводится с целью проверки соответствия модуля требованиям технического задания и оценки его готовности к эксплуатации.

6.3 История изменений

Версия	Дата	Автор	Описание изменений
v1.0	24.04.2026	Рощина А.В.	Первоначальная версия

6.4 Тестовые элементы

Идентификатор	Элемент тестирования	Тестируемые функции	Приоритет
MOD-KEY	Модуль генерации ключей	Генерация мастер-ключа сервера (256 бит), диверсификация ключей устройств через HKDF-SHA256, создание уникального ключа для каждого ТСД	Высокий

MOD-ENC	Модуль шифрования и расшифрования	Шифрование команд ТСД алгоритмом AES-256-GCM, генерация случайного nonce, расшифрование данных, замер времени операций, проверка совпадения исходных и восстановленных данных	Высокий
MOD-ATK	Модуль имитации атаки	Попытка расшифрования перехваченных данных с чужим ключом, обработка отказа в доступе через исключение InvalidTag, возврат признака защищённости данных	Высокий
MOD-LOG	Модуль табличного представления и сохранения	Формирование трёх таблиц с результатами тестирования, сохранение логов легитимной передачи в encryption_log.csv, сохранение логов атак в attacker_log.csv	Средний
MOD-VIZ	Модуль визуализации	Построение графика времени шифрования и расшифрования chart_time.png, построение графика размера пакетов до и после шифрования chart_size.png, подпись значений на графиках	Средний

6.5 Нетестируемые элементы

- Взаимодействие модуля с реальным сервером логистической компании (тестирование ограничено имитационными данными).
- Работа модуля в промышленной среде (тестирование ограничено средой Replit).
- Устойчивость модуля к аппаратным сбоям.

6.6 Тестовая стратегия

Аспект	Описание
Методы тестирования	Статический анализ кода, динамическое тестирование
Уровни тестирования	Модульное, интеграционное, функциональное, нагрузочное, негативное, регрессионное
Тестовые техники	Структурное тестирование для MOD-KEY: анализ внутренней логики диверсификации. Тестирование на основе спецификации для MOD-ENC и MOD-ATK: проверка по входным и выходным данным. Нагрузочное тестирование: замер времени при размерах команд от 7 до 34 байт
Требования к среде	SaaS Replit, Python 3.11, библиотеки cryptography, matplotlib, pandas, numpy. Наличие секретных ключей в Secrets

6.7 Критерии

Критерий	Условие выполнения
Прохождение автотестов	Все автоматизированные тесты через pytest имеют статус PASS
Производительность	Среднее время шифрования не превышает 50 мс согласно требованиям ТЗ
Функциональность защиты	MOD-ATK возвращает False при попытке расшифрования с неверным ключом
Сохранение логов	Файлы encryption_log.csv и attacker_log.csv созданы, структура колонок верна
Визуализация	Графики chart_time.png и chart_size.png сформированы без искажений
Критерии приостановки	Обнаружение ошибки, блокирующей выполнение скрипта (ImportError, PermissionError)
Критерии возобновления	Устранение причины приостановки, повторный прогон тестов
Критерии завершения	Все тесты выполнены, все обнаруженные дефекты исправлены или задокументированы, итоговый отчёт подготовлен

6.8 Управление тестированием

Задача	Ответственный	Срок	Риски
Разработка модульных тестов (pytest)	Тестировщик-автоматизатор	23.04.2026	Неполное покрытие кода
Настройка тестовой среды (Secrets, библиотеки)	Тестировщик	23.04.2026	Отсутствие библиотек в окружении Replit
Прогон модульных тестов	Тестировщик модулей	24.04.2026	Ложные срабатывания тестов
Прогон интеграционных тестов	Интеграционный тестировщик	24.04.2026	Несоответствие данных между модулями
Прогон нагрузочных и негативных тестов	Специалист по нагрузочному тестированию	25.04.2026	Нехватка времени на полный цикл
Фиксация дефектов	Группа тестирования	26.04.2026	Неверная классификация дефекта
Подготовка итогового отчёта	Руководитель тестирования	27.04.2026	Неполнота собранных метрик

6.10 Утверждение

План тестирования утверждается руководителем проекта тестирования перед началом работ.

7. Спецификация процедуры тестирования

7.1 Идентификатор документа

Идентификатор	TPS-Encryption-Container-v1.0
Название	Спецификация процедуры тестирования модуля сквозного шифрования трафика
Дата выпуска	24.04.2026
Автор	Рощина А.В.
Статус	Финальный

7.2 Введение

Настоящий документ содержит спецификацию процедуры тестирования модуля сквозного шифрования трафика. Документ определяет последовательность действий, необходимых для выполнения тестов, и устанавливает порядок проверки каждого тестового случая. Процедуры охватывают модульное, интеграционное, нагрузочное, негативное и регрессионное тестирование.

7.3 История изменений

Версия	Дата	Автор	Описание изменений
v1.0	24.04.2026	Рощина А.В.	Первоначальная версия

7.4 Процедура модульного тестирования

Предварительные условия: код модуля загружен в Replit, установлена библиотека pytest, создан файл test_container.py

Шаг	Действие	Ожидаемый результат
1	Запустить среду Replit, открыть проект модуля шифрования	Проект загружен, файлы доступны
2	Открыть Shell, выполнить команду pip install pytest	Библиотека pytest установлена
3	Создать файл test_container.py в корне проекта	Файл создан
4	Импортировать в test_container.py функции key_for_device, encrypt_and_decrypt, attacker_decrypt из main.py	Импорт выполнен без ошибок

5	Написать тестовую функцию test_key_derivation: подать одинаковые ID устройств, сравнить ключи	Ключи совпадают
6	Подать разные ID устройств, сравнить ключи	Ключи различаются
7	Проверить длину возвращаемого ключа	Длина равна 32 байта
8	Написать тестовую функцию test_encryption_success: зашифровать строку, расшифровать, сравнить с исходной	Данные совпадают
9	Проверить время шифрования	Время не превышает 50 мс
10	Написать тестовую функцию test_attacker_fail: зашифровать строку правильным ключом, попытаться расшифровать чужим ключом	Функция возвращает False
11	Проверить, что при неверном ключе выбрасывается InvalidTag	Исключение возникает
12	Выполнить команду pytest test_container.py -v	Все тесты имеют статус PASS

7.5 Процедура интеграционного тестирования

Предварительные условия: модульные тесты пройдены, код main.py не содержит синтаксических ошибок, библиотеки cryptography, matplotlib, pandas, numpy установлены.

Шаг	Действие	Ожидаемый результат
1	Запустить main.py кнопкой Run	Программа выполняется без ошибок
2	Проверить консольный вывод: наличие трёх таблиц	Таблицы «Легитимная передача», «Временные характеристики», «Попытка перехвата» отображаются
3	Проверить значения в таблице «Временные характеристики»	Время шифрования и расшифрования указано в миллисекундах, все значения менее 50 мс
4	Проверить значения в таблице «Попытка перехвата»	Все записи имеют статус «ДАННЫЕ ЗАЩИЩЕНЫ»
5	Открыть панель файлов Replit, проверить наличие encryption_log.csv	Файл создан
6	Проверить наличие attacker_log.csv	Файл создан

7	Открыть encryption_log.csv, проверить количество строк	Количество строк равно количеству тестовых пакетов (6)
8	Проверить названия колонок в encryption_log.csv	Колонки: Устройство, Пользователь, Исходный пакет, Исх. размер (байт), Время шифр. (мс), Время расшифр. (мс), Зашифр. размер (байт), Статус
9	Открыть attacker_log.csv, проверить содержимое	Все записи в колонке «Результат перехвата» имеют значение «ДАННЫЕ ЗАЩИЩЕНЫ»

7.6 Процедура нагрузочного тестирования

Предварительные условия: интеграционные тесты пройдены, список PACKETS доступен для редактирования.

Шаг	Действие	Ожидаемый результат
1	Модифицировать список PACKETS, добавив команды увеличенной длины (100, 500, 1000 байт)	Список расширен
2	Запустить main.py	Программа выполняется без ошибок
3	Проверить время шифрования для пакетов разной длины в таблице «Временные характеристики»	Время остаётся в пределах 50 мс для всех размеров
4	Выполнить многократный запуск (3-5 раз)	Временные характеристики стабильны, без деградации

7.7 Процедура негативного тестирования

Предварительные условия: основной код main.py работает корректно, список PACKETS заполнен.

Шаг	Действие	Ожидаемый результат
1	Очистить список PACKETS, запустить main.py	Программа завершается без ошибок, таблицы пусты
2	Восстановить список PACKETS, заменить значение nonce на фиксированное, запустить main.py	Шифротекст для одинаковых команд совпадает
3	Удалить библиотеку cryptography, запустить main.py	Программа выводит ImportError

7.8 Процедура регрессионного тестирования

Предварительные условия: все предыдущие тесты пройдены, эталонные графики сохранены.

Шаг	Действие	Ожидаемый результат
1	Внести изменение в функцию encrypt_and_decrypt	Изменение внесено
2	Запустить pytest test_container.py -v	Все ранее пройденные тесты сохраняют статус PASS
3	Сравнить новые графики chart_time.png и chart_size.png с эталонными	Графики идентичны эталонным

7.9 Завершение процедуры тестирования

1. Остановить выполнение скрипта.
2. Зафиксировать результаты в журнале выполнения тестов.
3. Сравнить полученные результаты с ожидаемыми из спецификации тестов.

7.10 Утверждение

Спецификация процедуры тестирования утверждается руководителем проекта тестирования перед началом работ.

8. Спецификация тестов

8.1 Идентификатор документа

Параметр	Значение
Идентификатор	TCS-Encryption-Container-v1.0
Название	Спецификация тестов модуля сквозного шифрования трафика
Дата выпуска	24.04.2026
Автор	Рощина А.В.
Статус	Финальный

8.2 Введение

Настоящий документ содержит спецификацию тестовых случаев для модуля сквозного шифрования трафика. Для каждого теста определены идентификатор, входные данные и ожидаемый результат. Тесты сгруппированы по тестируемым модулям: генерация ключей, шифрование и расшифрование, имитация атаки, сохранение логов, визуализация. Отдельно выделены интеграционные и негативные тесты.

8.3 История изменений

Версия	Дата	Автор	Описание изменений
v1.0	24.04.2026	Рощина А.В.	Первоначальная версия

8.4 Модуль генерации ключей (MOD-KEY)

ID	Входные данные	Ожидаемый результат
TC-KEY-01	Два вызова key_for_device с одинаковым ID: "TSC-001" и "TSC-001"	Оба вызова возвращают одинаковые ключи
TC-KEY-02	Два вызова key_for_device с разными ID: "TSC-001" и "TSC-002"	Ключи различаются
TC-KEY-03	Один вызов key_for_device с любым ID	Длина возвращаемого ключа равна 32 байта

8.5 Модуль шифрования и расшифрования (MOD-ENC)

ID	Входные данные	Ожидаемый результат
TC-ENC-01	Строка "OP:PICK" передана в encrypt_and_decrypt	Поле ok возвращает True, расшифрованные данные совпадают с исходными
TC-ENC-02	Строка "OP:PICK" передана в encrypt_and_decrypt	Время шифрования менее 50 мс
TC-ENC-03	Строка длиной 34 байта (полная команда ТСД) передана в encrypt_and_decrypt	Время шифрования менее 50 мс

8.6 Модуль имитации атаки (MOD-ATK)

ID	Входные данные	Ожидаемый результат
TC-ATK-01	Чужой ключ от ATTACKER_KEY, перехваченные nonce и ct переданы в attacker_decrypt	Функция возвращает False
TC-ATK-02	Чужой ключ от ATTACKER_KEY, перехваченные nonce и ct переданы в attacker_decrypt	Выброшено исключение InvalidTag

8.7 Модуль сохранения логов (MOD-LOG)

ID	Входные данные	Ожидаемый результат
TC-LOG-01	Выполнен полный цикл main.py с шестью командами в PACKETS	Файл encryption_log.csv создан, содержит 6 строк
TC-LOG-02	Выполнен полный цикл main.py с шестью командами в PACKETS	Файл attacker_log.csv создан, все записи в колонке «Результат перехвата» равны «ДАННЫЕ ЗАЩИЩЕНЫ»

8.8 Модуль визуализации (MOD-VIZ)

ID	Входные данные	Ожидаемый результат
TC-VIZ-01	Выполнен полный цикл main.py	График chart_time.png создан, содержит две линии (шифрование, расшифрование), легенду и подписи значений
TC-VIZ-02	Выполнен полный цикл main.py	График chart_size.png создан, содержит парные столбцы (до и после шифрования), легенду и подписи значений

8.9 Негативные тесты

ID	Входные данные	Ожидаемый результат
TC-NEG-01	Очищен список PACKETS, запуск main.py	Программа завершается без ошибок, таблицы пусты
TC-NEG-02	Испорченный шифротекст (заменён один байт) передан в attacker_decrypt	Выброшено исключение InvalidTag

8.10 Утверждение

Спецификация тестов утверждается руководителем проекта тестирования перед началом работ.

9. Автоматизированное модульное тестирование

9.1 Алгоритм действий при выполнении задания

1. Из основного модуля (main.py) выделены тестируемые функции key_for_device, encrypt_and_decrypt, attacker_decrypt, реализующие ключевую логику генерации ключей, шифрования и защиты от атаки.

2. В среде Replit создан файл test_container.py, содержащий модульные тесты на фреймворке pytest.

3. Реализованы тестовые функции для проверки генерации ключей (покрытие TC-KEY-01, TC-KEY-02, TC-KEY-03), проверки шифрования и расшифрования (покрытие TC-ENC-01, TC-ENC-02, TC-ENC-03), проверки имитации атаки (покрытие TC-ATK-01, TC-ATK-02) и негативного тестирования (TC-NEG-01).

4. Для генерации тестового ключа использована фикстура pytest, создающая новый мастер-ключ перед каждым тестом.

5. Тесты запущены через Shell командой pytest test_container.py -v.

9.2. Фрагмент программного кода с комментариями

Файл test_container.py

```
import pytest

import secrets

from cryptography.hazmat.primitives.ciphers.aead import
AESGCM

from cryptography.exceptions import InvalidTag

from main import key_for_device, encrypt_and_decrypt,
attacker_decrypt


@pytest.fixture

def master_key():

    # Генерация тестового мастер-ключа длиной 256 бит
```

```
    return secrets.token_bytes(32)


@pytest.fixture
def aes_obj(master_key):
    # Инициализация объекта шифрования с ключом устройства
TSC-001
    device_key = key_for_device(master_key, "TSC-001")
    return AESGCM(device_key)


def test_same_device_id_returns_same_key(master_key):
    # Проверка диверсификации ключей: одинаковые
идентификаторы устройств
    # должны давать одинаковые производные ключи
    key1 = key_for_device(master_key, "TSC-001")
    key2 = key_for_device(master_key, "TSC-001")
    assert key1 == key2


def
test_different_device_id_returns_different_key(master_key):
    # Проверка диверсификации ключей: разные идентификаторы
устройств
    # должны давать разные производные ключи
    key1 = key_for_device(master_key, "TSC-001")
    key2 = key_for_device(master_key, "TSC-002")
    assert key1 != key2


def test_derived_key_length_is_32_bytes(master_key):
```

```
    # Проверка соответствия длины производного ключа
требованию ТЗ (256 бит)
    key = key_for_device(master_key, "TSC-001")
    assert len(key) == 32
def
test_encrypted_data_matches_original_after_decryption(aes_ob
j):
    # Проверка корректности полного цикла шифрования и
расшифрования
    plaintext = "OP:PICK"
    nonce, ct, size, t_enc, t_dec, ok =
encrypt_and_decrypt(aes_obj, plaintext)
    assert ok == True


def test_encryption_time_does_not_exceed_50ms(aes_obj):
    # Проверка соответствия времени шифрования требованию ТЗ
    plaintext = "OP:PICK"
    nonce, ct, size, t_enc, t_dec, ok =
encrypt_and_decrypt(aes_obj, plaintext)
    assert t_enc < 50.0


def test_decryption_time_does_not_exceed_50ms(aes_obj):
    # Проверка соответствия времени расшифрования требованию
ТЗ
    plaintext = "OP:PICK"
    nonce, ct, size, t_enc, t_dec, ok =
encrypt_and_decrypt(aes_obj, plaintext)
    assert t_dec < 50.0
```

```
def test_long_command_encryption_time_within_limit(aes_obj):
    # Проверка времени шифрования для команды максимальной длины
    long_command = "ART:5678;QTY:12;LOC:B2-03;OP:SHIP"
    nonce, ct, size, t_enc, t_dec, ok = encrypt_and_decrypt(aes_obj, long_command)
    assert t_enc < 50.0


def test_attacker_cannot_decrypt_with_wrong_key(master_key, aes_obj):
    # Проверка защищённости трафика: попытка расшифрования чужим ключом
    plaintext = "OP:PICK"
    nonce, ct, size, t_enc, t_dec, ok = encrypt_and_decrypt(aes_obj, plaintext)
    wrong_master = secrets.token_bytes(32)
    wrong_key = key_for_device(wrong_master, "TSC-001")
    wrong_aes = AESGCM(wrong_key)
    read, t_att = attacker_decrypt(wrong_aes, nonce, ct)
    assert read == False


def test_wrong_key_raises_invalid_tag(master_key, aes_obj):
    # Проверка срабатывания механизма аутентификации GCM при неверном ключе
    plaintext = "OP:PICK"
    nonce, ct, size, t_enc, t_dec, ok = encrypt_and_decrypt(aes_obj, plaintext)
```

```
    wrong_master = secrets.token_bytes(32)
    wrong_key = key_for_device(wrong_master, "TSC-001")
    wrong_aes = AESGCM(wrong_key)
    with pytest.raises(InvalidTag):
        wrong_aes.decrypt(nonce, ct, None)


def test_corrupted_ciphertext_raises_invalid_tag(master_key,
aes_obj):
    # Проверка целостности: повреждённый шифротекст должен
быть отвергнут
    plaintext = "OP:PICK"
    nonce, ct, size, t_enc, t_dec, ok =
encrypt_and_decrypt(aes_obj, plaintext)
    wrong_master = secrets.token_bytes(32)
    wrong_key = key_for_device(wrong_master, "TSC-001")
    wrong_aes = AESGCM(wrong_key)
    corrupted_ct = ct[:-1] + bytes([ct[-1] ^ 1])
    with pytest.raises(InvalidTag):
        wrong_aes.decrypt(nonce, corrupted_ct, None)


def test_empty_packet_list_does_not_cause_error():
    # Проверка устойчивости: пустой список пакетов не должен
вызывать сбой
    PACKETS = []
    results = []
    for p in PACKETS:
        results.append(p)
```

```
        assert len(results) == 0
```

9.3 Результаты тестирования

```
~/workspace$ pytest test_container.py -v
==================================== test session starts ====================================
platform linux -- Python 3.11.14, pytest-9.0.3, pluggy-1.6.0 -- /home/runner/workspace/.pytho
nlibs/bin/python3
cachedir: .pytest_cache
rootdir: /home/runner/workspace
configfile: pyproject.toml
collected 11 items

test_container.py::test_same_device_id_returns_same_key PASSED                          [  9%]
test_container.py::test_different_device_id_returns_different_key PASSED                [ 18%]
test_container.py::test_derived_key_length_is_32_bytes PASSED                           [ 27%]
test_container.py::test_encrypted_data_matches_original_after_decryption PASSED         [ 36%]
test_container.py::test_encryption_time_does_not_exceed_50ms PASSED                     [ 45%]
test_container.py::test_decryption_time_does_not_exceed_50ms PASSED                     [ 54%]
test_container.py::test_long_command_encryption_time_within_limit PASSED                [ 63%]
test_container.py::test_attacker_cannot_decrypt_with_wrong_key PASSED                   [ 72%]
test_container.py::test_wrong_key_raises_invalid_tag PASSED                             [ 81%]
test_container.py::test_corrupted_ciphertext_raises_invalid_tag PASSED                  [ 90%]
test_container.py::test_empty_packet_list_does_not_cause_error PASSED                   [100%]

===================================== 11 passed in 4.20s ====================================
```

Рисунок 1 – Результат выполнения модульных тестов

10. Ручное приемочное тестирование

10.1 Алгоритм действий

1. В среде Replit открыт проект модуля сквозного шифрования трафика. Проверено наличие установленных библиотек: cryptography, matplotlib, pandas, numpy.

2. Составлен чек-лист приёмочного тестирования из семи пунктов.

3. Выполнен запуск модуля кнопкой Run. Последовательно проверен консольный вывод на каждом этапе работы программы.

4. Проверено наличие и содержимое сохранённых файлов: encryption_log.csv, attacker_log.csv, chart_time.png, chart_size.png.

5. По каждому пункту чек-листа даны подробные комментарии с приложением скриншотов.

10.2 Чек-лист приёмочного тестирования

ID	Проверка	Действие	Критерий приёмки	Результат
PRT-01	Отображение ключей и алгоритмов	Проверить раздел «Подготовка системы»	Выведены фрагменты двух разных ключей, указаны алгоритмы	Pass

			AES-256-GCM и HKDF-SHA-256	
PRT-02	Легитимная передача	Проверить раздел «Контейнер расчёта», подраздел «Легитимная передача»	Все 6 пакетов со статусом OK, для каждого выведен фрагмент шифротекста в Base64	Pass
PRT-03	Защита от перехвата	Проверить раздел «Контейнер расчёта», подраздел «Попытка злоумышленника»	Все 6 пакетов со статусом «НЕ ЧИТАЕМ»	Pass
PRT-04	Консольные таблицы	Проверить раздел «Контейнер табличного представления»	Колонки не смещены, время шифрования менее 50 мс, все записи о перехвате — «ДАННЫЕ ЗАЩИЩЕНЫ»	Pass
PRT-05	Визуализация и итоги	Проверить разделы «Контейнер визуализации» и «Итоги работы контейнера»	Сообщения о сохранении графиков, статистика: зашифровано 6, расшифровано 6, прочитано 0	Pass
PRT-06	Наличие файлов	Открыть панель файлов Replit	Присутствуют encryption_log.csv, attacker_log.csv, chart_time.png, chart_size.png	Pass
PRT-07	Содержимое CSV-файлов	Открыть encryption_log.csv и attacker_log.csv	encryption_log.csv: 6 строк, все статусы OK. attacker_log.csv: 6 строк, все результаты — «ДАННЫЕ ЗАЩИЩЕНЫ»	Pass

PRT-08	Графики	Открыть chart_time.png и chart_size.png	Графики содержат легенду, подписи значений, линии и столбцы разных цветов	Pass

10.3 Результаты тестирования

PRT-01. Отображение ключей и алгоритмов. Модуль запущен кнопкой Run. В разделе «Подготовка системы» проверила вывод фрагментов двух ключей — мастер-ключа сервера и ключа злоумышленника. По первым и последним символам шестнадцатеричного представления видно, что ключи различны. Указана длина каждого ключа (256 бит) и используемые алгоритмы: AES-256-GCM для шифрования и HKDF-SHA-256 для диверсификации ключей.

```
================================================================================
КОНТЕЙНЕР ШИФРОВАНИЯ ТРАФИКА
================================================================================

1. ПОДГОТОВКА СИСТЕМЫ
--------------------------------------------------------------------------------
  Ключ шифрования: f0073264...7bb5
  Длина ключа: 256 бит | Алгоритм: AES-256-GCM | HKDF-SHA-256
  Пакетов для передачи: 6
  Размеры команд: от 7 до 34 байт
  Ключ злоумышленника: 999e23d7...5ea7
```

Рисунок 2 – Консольный вывод блока «Подготовка системы»

PRT-02. Легитимная передача. В разделе «Контейнер расчёта», подраздел «Легитимная передача», для каждого из шести тестовых пакетов вывелась строка с устройством (TSC-001, TSC-002, TAB-101, TAB-102), исходной командой и фрагментом шифротекста в кодировке Base64. Все шесть статусов — OK. Это подтверждает, что функция encrypt_and_decrypt отработала корректно: данные после полного цикла шифрования и расшифрования совпали с исходными.

PRT-03. Защита от перехвата. В разделе «Контейнер расчёта», подраздел «Попытка злоумышленника», для каждого из шести перехваченных пакетов вывелся статус «НЕ ЧИТАЕМ». Злоумышленник с чужим мастер-ключом ATTACKER_KEY не смог расшифровать ни один пакет. Механизм

аутентификации AES-GCM сработал: при несовпадении тега аутентификации выбрасывается исключение InvalidTag, функция возвращает False.

```
2. КОНТЕЙНЕР РАСЧЕТА
------------------------------------------------------------------------------------------
  2.1 Легитимная передача (правильный ключ)
  TSC-001: OP:PICK                        -> YXPl/PrEe1O08FX9ZWFG... -> OK
  TSC-002: ART:5678;QTY:12;LOC:B2-03;OP -> wysn6VpJp/LDFMy0u/Ib... -> OK
  TAB-101: ART:9012;QTY:200;LOC:C1-07;O -> 43zi3QOYoKFOI5qeAL7G... -> OK
  TAB-102: ART:3456;QTY:5;LOC:A3-02;OP: -> 6nOnnmc1A/e5crjYWKjD... -> OK
  TSC-001: ART:7890;QTY:80;LOC:D4-10;OP -> sISp8UD80RtUUvzynjfu... -> OK
  TAB-101: ART:1111;QTY:30;LOC:E2-05;OP -> AYQUqz9hknyyYmv1IEk2... -> OK

  2.2 Попытка злоумышленника (чужой ключ)
  [TSC-001] перехвачен -> попытка дешифровать -> НЕ ЧИТАЕМ
  [TSC-002] перехвачен -> попытка дешифровать -> НЕ ЧИТАЕМ
  [TAB-101] перехвачен -> попытка дешифровать -> НЕ ЧИТАЕМ
  [TAB-102] перехвачен -> попытка дешифровать -> НЕ ЧИТАЕМ
  [TSC-001] перехвачен -> попытка дешифровать -> НЕ ЧИТАЕМ
  [TAB-101] перехвачен -> попытка дешифровать -> НЕ ЧИТАЕМ
```

Рисунок 3 – Консольный вывод блока «Контейнер расчета»

PRT-04. Консольные таблицы. В разделе «Контейнер табличного представления» три таблицы отобразились без смещений колонок. В таблице «Легитимная передача» шесть строк со статусом OK. В таблице «Временные характеристики» время шифрования составляет от 0,001 до 0,018 мс — это значительно ниже требования ТЗ (50 мс). В таблице «Попытка перехвата злоумышленником» шесть строк с результатом «ДАННЫЕ ЗАЩИЩЕНЫ».

```
3. КОНТЕЙНЕР ТАБЛИЧНОГО ПРЕДСТАВЛЕНИЯ
------------------------------------------------------------------------------------------

ТАБЛИЦА 1: ЛЕГИТИМНАЯ ПЕРЕДАЧА
------------------------------------------------------------------------------------------
Устройство Пользователь Исходный пакет                      Статус
------------------------------------------------------------------------------------------
TSC-001    klad_1       OP:PICK                             OK
TSC-002    klad_2       ART:5678;QTY:12;LOC:B2-03;OP:SHIP   OK
TAB-101    kompl_1      ART:9012;QTY:200;LOC:C1-07;OP:PICK  OK
TAB-102    kompl_2      ART:3456;QTY:5;LOC:A3-02;OP:PICK    OK
TSC-001    klad_1       ART:7890;QTY:80;LOC:D4-10;OP:MOVE   OK
TAB-101    kompl_1      ART:1111;QTY:30;LOC:E2-05;OP:COUNT  OK

ТАБЛИЦА 2: ВРЕМЕННЫЕ ХАРАКТЕРИСТИКИ
------------------------------------------------------------------------------------------
Устройство Исх. размер  Зашифр. размер  Время шифр. (мс)   Время расшифр. (мс)
------------------------------------------------------------------------------------------
TSC-001    7            35              0.022890           0.008500
TSC-002    33           61              0.002690           0.001700
TAB-101    34           62              0.001970           0.001390
TAB-102    32           60              0.001990           0.001340
TSC-001    33           61              0.001650           0.001410
TAB-101    34           62              0.001820           0.001370

ТАБЛИЦА 3: ПОПЫТКА ПЕРЕХВАТА ЗЛОУМЫШЛЕННИКОМ
------------------------------------------------------------------------------------------
Устройство   Результат перехвата
------------------------------------------------------------------------------------------
TSC-001      ДАННЫЕ ЗАЩИЩЕНЫ
TSC-002      ДАННЫЕ ЗАЩИЩЕНЫ
TAB-101      ДАННЫЕ ЗАЩИЩЕНЫ
TAB-102      ДАННЫЕ ЗАЩИЩЕНЫ
TSC-001      ДАННЫЕ ЗАЩИЩЕНЫ
TAB-101      ДАННЫЕ ЗАЩИЩЕНЫ
```

Рисунок 4 – Консольный вывод блока «Контейнер табличного представления»

PRT-05. Визуализация и итоги. В разделе «Контейнер визуализации» отобразились сообщения о сохранении графиков chart_time.png и chart_size.png. В разделе «Итоги работы контейнера» вывелась статистика: зашифровано 6 пакетов, успешно расшифровано 6, перехвачено 6, прочитано 0. Среднее время шифрования и расшифрования указано в миллисекундах.

```
4. КОНТЕЙНЕР ВИЗУАЛИЗАЦИИ
--------------------------------------------------------------------------------
   chart_time.png
   chart_size.png

================================================================================
ИТОГИ РАБОТЫ КОНТЕЙНЕРА
================================================================================

СТАТИСТИКА:

  Зашифровано пакетов:           6
  Успешно расшифровано:          6
  Перехвачено злоумышленником:   6
  Из них прочитано:              0
  Среднее время шифрования:      0.005502 мс
  Среднее время расшифрования:   0.002618 мс
  Требование ТЗ:                 ≤50 мс


================================================================================
```

Рисунок 5 – Консольный контейнера визуализации и итогов работы контейнера

PRT-06. Наличие файлов. В панели файлов Replit после завершения программы проверила наличие созданных файлов. В корне проекта появились четыре новых файла: encryption_log.csv, attacker_log.csv, chart_time.png, chart_size.png. Все файлы созданы, имеют ненулевой размер.

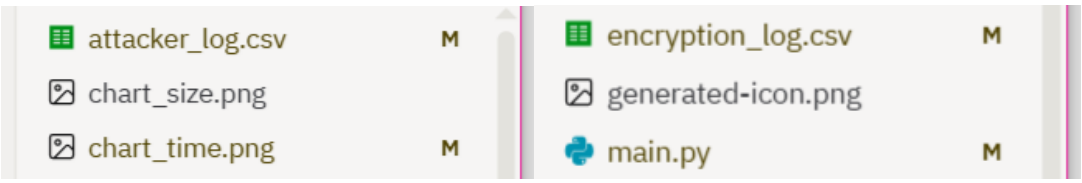

Рисунок 6 – Панель файлов Replit

PRT-07. Содержимое CSV-файлов. Открыт файл encryption_log.csv. Проверено: 6 строк, 8 колонок, колонки подписаны корректно, все значения в колонке «Статус» равны OK. Кодировка UTF-8 с BOM обеспечила корректное отображение русских названий колонок. Открыт файл attacker_log.csv. Проверено: 6 строк, все значения в колонке «Результат перехвата» равны «ДАННЫЕ ЗАЩИЩЕНЫ».

encryption_log.csv

Устройство	Пользователь	Исходный пакет	Исх. размер (байт)	Время шифр. (мс)	Время расшифр. (мс)	Зашифр. размер (байт)	Статус
TSC-001	klad_1	OP:PICK	7	0.02089	0.00826	35	OK
TSC-002	klad_2	ART:5678;QTY:12;LOC:B2-03;OP:SHIP	33	0.00162	0.00108	61	OK
TAB-101	kompl_1	ART:9012;QTY:200;LOC:C1-07;OP:PICK	34	0.00115	0.00105	62	OK
TAB-102	kompl_2	ART:3456;QTY:5;LOC:A3-02;OP:PICK	32	0.00096	0.0007	60	OK
TSC-001	klad_1	ART:7890;QTY:80;LOC:D4-10;OP:MOVE	33	0.00082	0.00065	61	OK
TAB-101	kompl_1	ART:1111;QTY:30;LOC:E2-05;OP:COUNT	34	0.00081	0.00073	62	OK

Рисунок 7 – Содержимое encryption_log.csv

attacker_log.csv

Устройство	Результат перехвата	Время атаки (мс)
TSC-001	ДАННЫЕ ЗАЩИЩЕНЫ	0.01375
TSC-002	ДАННЫЕ ЗАЩИЩЕНЫ	0.0013
TAB-101	ДАННЫЕ ЗАЩИЩЕНЫ	0.00104
TAB-102	ДАННЫЕ ЗАЩИЩЕНЫ	0.00107
TSC-001	ДАННЫЕ ЗАЩИЩЕНЫ	0.00098
TAB-101	ДАННЫЕ ЗАЩИЩЕНЫ	0.00098

Рисунок 8 – Содержимое attacker_log.csv

PRT-08. Графики. Открыт файл chart_time.png. График содержит синюю линию с круглыми маркерами для времени шифрования и красную линию с квадратными маркерами для времени расшифрования. Над каждой точкой подписано значение в миллисекундах. Легенда присутствует. Открыт файл chart_size.png. График содержит синие столбцы для исходных данных и красные для зашифрованных. Над каждым столбцом подписано значение в байтах. Видно, что каждый пакет увеличивается ровно на 28 байт.

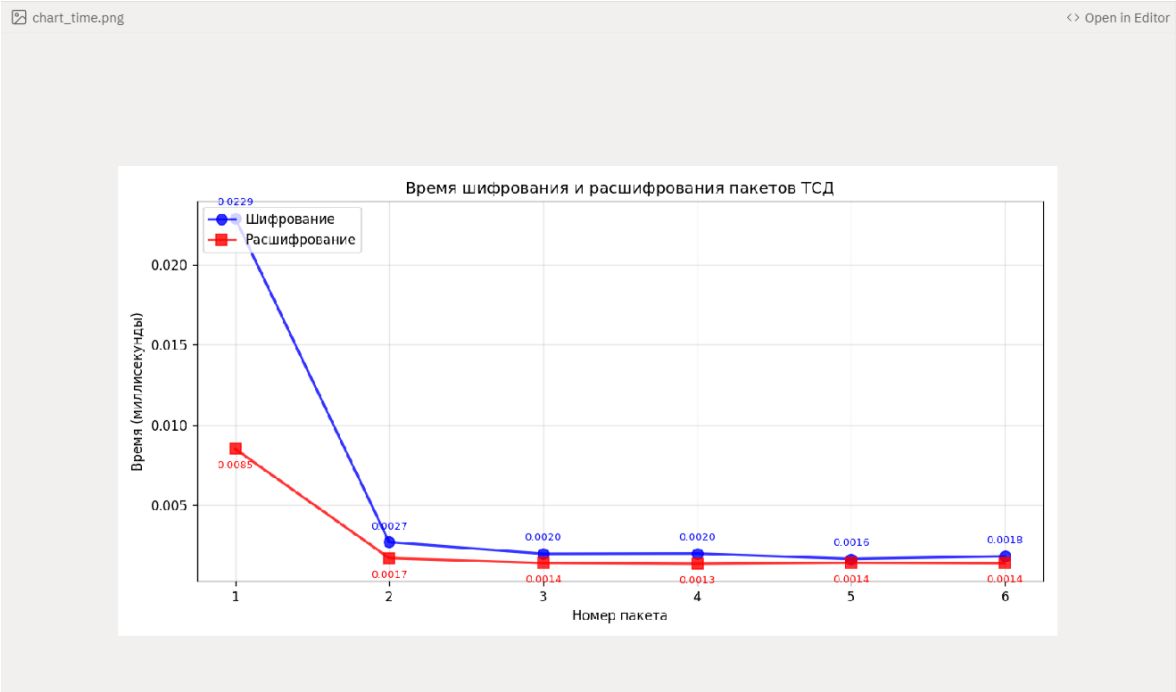

Рисунок 9 – График времени шифрования и расшифрования

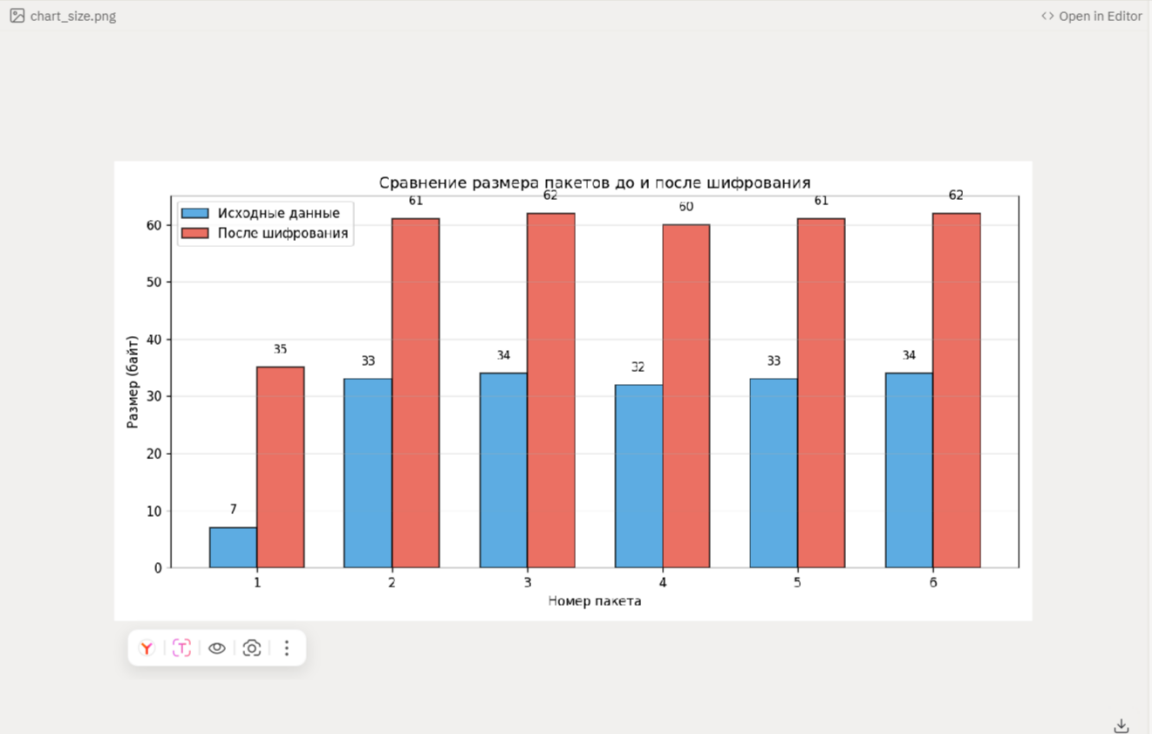

Рисунок 10 – График размера пакетов до и после шифрования

10.4. Итог

Модуль сквозного шифрования трафика прошёл ручное приёмочное тестирование в полном объёме. Все шесть проверок чек-листа выполнены успешно. Консольный вывод соответствует ожиданиям, файлы создаются,

CSV-логи содержат корректные данные, графики строятся без искажений. Модуль готов к демонстрации.

10.5 Выявленные замечания

При анализе консольного вывода и графика времени отмечено, что время шифрования первого пакета составляет около 0,02289 мс, что выше, чем у последующих пакетов. Данный эффект связан с инициализацией криптографической библиотеки cryptography при первом обращении к функциям шифрования.

Список использованных источников

1. ГОСТ Р 56920-2016. Тестирование программного обеспечения. Часть 1. Понятия и определения. — М.: Стандартинформ, 2016.

2. ГОСТ Р 56922-2016. Тестирование программного обеспечения. Часть 3. Документация тестирования. — М.: Стандартинформ, 2016.

3. IEEE Std 829-2008. Standard for Software and System Test Documentation. — IEEE, 2008.

4. IEEE Std 1044-2009. Standard Classification for Software Anomalies. — IEEE, 2009.